

PROMETEO



**Aplicación multiplataforma
para la gestión digital del mantenimiento de vehículos**

Desarrollo Aplicaciones Multiplataforma - Online

Alumno: Álvaro Fco. Medina Rodriguez

Tutora: Luz María Álvarez Moreno

PROMETEO

A mi padre, por ayudarme en todo.

A mi madre, por estar ahí siempre.

A mi Leoncilla, por hacerme mejor.

A mi Juan, por darme otra motivación más.

*Y a mi yo del pasado, porque a pesar de
tantas vueltas, al final lo hicimos.*

Esto es solo el principio.

Os quiero mucho.

PROMETEO

Índice

Índice	3
Abstract	6
Abstract (ES)	6
Abstract (EN)	6
Justificación Del Proyecto	7
Tabla 1	8
Introducción	8
Objetivos	9
Descripción	13
Arquitectura De La Solución	14
Figura 1	14
Casos De Uso	14
Figura 2	15
Tabla 2	16
Tabla 3	16
Tabla 4	16
Tabla 5	17
Tabla 6	17
Tabla 7	18
Tabla 8	18
Tabla 9	18
Tabla 10	19
Tabla 11	19
Tabla 12	19
Diseños	20
Diagrama De Clases	20
Figura 3	20
Diagrama Entidad Relación	21
Figura 4	21
Diagrama De La Base De Datos	22
Figura 5	23
Seguridad Y Control De Acceso: Row Level Security	25
Políticas Aplicadas Por Tabla	25
Tabla 13	25
Tabla 14	26
Tabla 15	27
Políticas De Storage	27
Impacto En La Arquitectura De La Aplicación	28
Diagrama De Flujo De Navegación	28
Figura 6	28
Figura 7	30

PROMETEO

	Figura 8	31
	Figura 9	31
	Figura 10	32
	Figura 11	32
	Figura 12	33
	Figura 13	33
Interfaces		34
	Tabla 16	35
	Tabla 17	36
	Figura 14	36
	Figura 15	37
	Figura 16	38
	Figura 17	38
	Figura 18	39
	Figura 19	39
	Figura 20	40
	Figura 21	40
	Figura 22	40
	Figura 23	41
	Figura 24	41
Prototipo interactivo		41
	Figura 25	42
Diagrama De Red		43
	Figura 26	43
Tecnología		44
Dart		44
	Figura 27	44
Flutter		45
	Figura 28	45
Supabase		45
	Figura 29	45
PostgreSQL		46
	Figura 30	46
Git		46
	Figura 31	46
Github		47
	Figura 32	47
Visual Studio Code		47
	Figura 33	47
Mailtrap		48
	Figura 34	48
Emuladores (de dispositivos móviles).		49

PROMETEO

Integración Flutter + Supabase	49
Figura 35	49
Metodología	50
Diagrama de Gantt	51
Figura 36.1	51
Figura 36.2	52
Figura 36.3	53
Figura 36.4	54
Plan de pruebas (Testing)	54
Pruebas Unitarias	55
Pruebas de Caja Negra	55
Tabla 18	56
Pruebas de Caja Blanca	58
Tabla 19	58
Pruebas de integración y sistema	59
Presupuesto	60
Tabla 20	60
Trabajos Futuros	60
Conclusiones	61
Referencias	62

Abstract

Abstract (ES)

El objetivo de este proyecto es desarrollar una aplicación multiplataforma que permita digitalizar el mantenimiento de vehículos sin importar la tipología, mejorando la organización de la información y facilitando la consulta en cualquier momento. Se ha desarrollado con Flutter, para simplificar el despliegue multiplataforma con un backend en Supabase para gestionar la base de datos, autenticación de usuarios y almacenamiento de información. Para la arquitectura del sistema se ha optado por cliente-servidor, con especial énfasis en la usabilidad, escalabilidad y reutilización de código para las diferentes plataformas.

La aplicación permite a los usuarios registrarse, gestionar su cuenta, añadir vehículos y registrar intervenciones asociadas, incluyendo detalles sobre estas. Se ha usado una metodología incremental, y para la parte visual (UX/UI), se ha optado por Figma como herramienta para las interfaces y prototipos.

Abstract (EN)

This project aim is to develop a cross-platform app, enabling the digital tracking of vehicles maintenances, regardless of its type, thereby improving info organization and easing the access. Flutter has been the chosen language to simplify the deployment, using Supabase as a backend to the database, user authentication and data storage. The chosen architecture is Client-Server, because of the focus on usability, scalability and code reusability across different platforms.

In the app, users can register, manage their account, add vehicles, and record associated maintenance tasks, with related details. An incremental methodology was used, and for UX & UI, Figma has been used.

PROMETEO

Justificación Del Proyecto

La principal motivación del proyecto surge de una necesidad real e identificable: Tener un historial completo y centralizado en un solo lugar de todas las intervenciones que se realicen a cualquiera de los vehículos de un hogar, ya sean en taller o en casa. Ya existen algunas aplicaciones de este estilo, aunque se han detectado diferentes tipos de limitaciones:

- Para vehículos relativamente modernos (10 años o menos aprox).
- Para vehículos de una sola marca.
- Para un único tipo de vehículo.
- Solo permiten la visualización de intervenciones selladas (las incluyen terceras personas), sin incluir costes.
- No se registran todos los tipos de intervenciones.

Este proyecto busca ofrecer una solución más flexible y generalista, permitiendo:

- Registrar diferentes tipos de vehículos, independientemente de marcas, motorización, etc...
- Introducir intervenciones realizadas tanto en talleres oficiales como en casa por el usuario.
- Adjuntar documentación asociada, si se desea.
- Consultar el historial completo, de forma cronológica o por tipología de intervención.

El centrar el proyecto en usuarios particulares permite un alcance realista, en base al nivel y duración del proyecto, pero dejando abiertas diferentes vías para ampliaciones futuras.

Tabla 1*Comparación con soluciones existentes*

	Papel	Adjuntos	App de Notas (GKeep, Notion,...)	Apps Especializadas	ESTE PROYECTO
Acceso Multiplataforma	✗	✓	✓	✓	✓
Facilidad Organización	✗	⚠	⚠	✓	✓
Búsqueda rápida info	✗	✗	⚠	✓	✓
Historial completo	⚠	⚠	⚠	⚠	✓
Múltiples vehículos (y tipos)	✓ ✓	✓ ✓	✓ ✓	⚠ ⚠	✓ ✓

*LEYENDA: ✓ Lo tiene / ⚠ Depende de como lo organice / ✗ No lo tiene

Introducción

De un tiempo a esta parte hemos visto como muchas industrias tradicionales han iniciado su transformación hacia el mundo digital, aunque ciertos sectores se han quedado algo atrasados, o ni siquiera se han planteado el cambio. Hablamos, en este caso, del mantenimiento de vehículos, cuyo seguimiento se hace de forma poco estructurada, ya que cada taller y cada propietario es un mundo: Desde los más tradicionales que siguen con el papel (ya sean facturas, anotaciones en el libro de mantenimiento, etc...), hasta los que empiezan a asomarse a las tecnologías (envío de facturas por medios electrónicos, dueños que hacen seguimientos en aplicaciones de notas, etc...). El tener diversas fuentes de información hace que el seguimiento del historial del vehículo sea complicado, es de ahí de donde nace este proyecto, junto con el feedback recibido con la encuesta previa realizada antes de empezar a desarrollar este proyecto.

En un primer momento el cliente objetivo serán los usuarios particulares, aunque desde un enfoque genérico y escalable, para que en el futuro se puedan incluir nuevos

PROMETEO

perfiles de usuario y otras funcionalidades avanzadas, sin afectar en demasía a la arquitectura.

Este proyecto propone una aplicación multiplataforma (web y app móvil) para que actúe como un libro digital de intervenciones donde tener una relación completa, sencilla, accesible y organizada de éstas. Al centralizarlo todo en un solo lugar, se simplifica también el seguimiento en caso de tener varios vehículos, sin importar de qué tipo sean, y permite añadir operaciones realizadas fuera de talleres, para tener un mejor respaldo en caso de una potencial venta futura del vehículo. A todo esto se suma también que hay profesionales que todavía no disponen de sistemas digitales propios.

Objetivos

El sistema tiene como objetivo permitir a los usuarios gestionar el mantenimiento de sus vehículos mediante una aplicación multiplataforma, registrando intervenciones, consultando su historial y almacenando información relevante al respecto:

R01 - El sistema debe permitir a los usuarios registrarse, iniciar sesión y gestionar su cuenta.

R01F01 - El sistema permitirá a los usuarios crear su cuenta.

R01F01T01 - Crear tabla USUARIO en la base de datos.

R01F01T01P01 - Insertar un usuario de prueba en la BBDD.

R01F01T02 - Implementar formulario de registro en la app.

R01F01T02P01 - Mostrar la pantalla de registro.

R01F01T03 - Validar email y contraseña.

R01F01T03P01 - Comprobar la validación de ambos campos.

R01F01T04 - Integrar el registro con Supabase Auth.

R01F01T04P01 - Registro exitoso en Supabase.

R01F02 - El sistema debe permitir al usuario iniciar sesión.

R01F02T01 - Implementar formulario de Login.

PROMETEO

R01F02T01P01 - Mostrar pantalla Login.

R01F02T02 - Validar credenciales.

R01F02T02P01 - Validar Login (Correcto/Incorrecto).

R01F02T03 - Integrar autenticación con Supabase.

R01F02T03P01 - Acceso a la app con login correcto.

R01F03 - El sistema permite modificar y eliminar la cuenta.

R01F03T01 - Mostrar datos usuario.

R01F03T01P01 - Mostar datos correctamente.

R01F03T02 - Permitir edición de datos.

R01F03T02P01 - Guardar cambios.

R01F03T03 - Implementar eliminación de cuenta.

R01F01T03P01 - Eliminar usuario.

R02 - El sistema debe permitir a los usuarios gestionar sus vehículos.

R02F01 - El sistema debe permitir registrar un vehículo.

R02F01T01 - Crear tabla VEHICULO en la base de datos.

R02F01T1P01 - Insertar vehículo en la BBDD.

R02F01T02 - Crear formulario de alta de vehículos.

R02F01T02P01 - Mostrar formulario.

R02F01T03 - Relacionar vehículo con usuario

R02F01T03P01 - Asociación correcta usuario-vehículo.

R02F02 - El sistema debe mostrar los vehículos del usuario.

R02F02T01 - Consultar vehículos del usuario.

R02F02T01P01 - Consultar correcta.

R02F02T02 - Mostrar lista en la aplicación.

R02F02T02P01 - Visualizar la lista de vehículos.

R02F03 - El sistema debe permitir modificar un vehículo.

PROMETEO

R02F03T01 - Implementar edición.

R02F03T01P01 - Mostrar formulario con datos originales.

R02F03T02 - Actualizar datos en la BBDD.

R02F03T02P01 - Guardar cambios.

R02F04 - El sistema debe permitir eliminar un vehículo.

R02F04T01 - Implementar eliminación de vehículos.

R02F04T01P01 - Confirmar eliminación.

R02F04T02 - Eliminar de la BBDD.

R02F04T02P01 - Eliminación correcta

R03 - El sistema debe permitir a los usuarios registrar y consultar intervenciones de sus vehículos.

R03F01 - El sistema debe permitir registrar una intervención.

R03F01T01 - Crear tabla INTERVENCION en la base de datos.

R03F01T1P01 - Insertar intervención en la BBDD.

R03F01T02 - Crear formulario de alta de intervenciones.

R03F01T02P01 - Mostrar formulario.

R03F01T03 - Relacionar intervenciones con vehículo

R03F01T03P01 - Asociación correcta vehículo-intervención.

R03F02 - El sistema debe mostrar el historial de intervenciones.

R03F02T01 - Consultar intervenciones por vehículo.

R03F02T01P01 - Consultar correcta.

R03F02T02 - Mostrar lista cronológica de más reciente a más antigua.

R03F02T02P01 - Ordenación y visualización correcta.

R03F03 - El sistema debe permitir modificar una intervención.

R03F03T01 - Implementar edición.

R03F03T01P01 - Mostrar formulario con datos originales.

PROMETEO

R03F03T02 - Actualizar datos en la BBDD.

R03F03T02P01 - Guardar cambios.

R03F04 - El sistema debe permitir eliminar una intervención.

R03F04T01 - Implementar eliminación de intervenciones.

R03F04T01P01 - Confirmar eliminación.

R03F04T02 - Eliminar de la BBDD.

R03F04T02P01 - Eliminación correcta.

R04 - El sistema debe garantizar la seguridad de los datos del usuario.

R04F01 - El login debe realizarse de forma segura.

R04F01T01 - Implementación de login seguro.

R04F01T01P01 - Acceso controlado.

R04F01T02 - Gestión de sesiones.

R04F01T02P01 - Log de sesiones.

R04F02 - Control de acceso.

R04F02T01 - Configuración de Row Level Security (RLS).

R04F02T01P01 - Usuario no accede a datos ajenos.

R04F02T02 - Liminar acceso por usuario.

R05 - El sistema debe permitir almacenar documentos relacionados con las intervenciones.

R05F01 - El sistema debe permitir la subida de archivos.

R05F01T01 - Configurar el almacenamiento.

R05F01T02 - Subir archivo.

R05F01T02P01 - Subida correcta.

R05F01T03 - Guardar URL.

R05F01T03P01 - Guardar URL en la BBDD.

R06 - El sistema debe funcionar tanto en web como en dispositivos móviles

R06F01 - Aplicación Flutter

PROMETEO

R06F01T01 - Configurar proyecto Flutter

R06F01T01P01 - App ejecuta en Android

R06F01T01P02 - App ejecuta en iOS

R06F01T01P03 - App ejecuta en web

R06F01T02 - Diseñar Interfaces

R06F01T03 - Integrar API

R06F01T03P01 - Conexión con backend

Descripción

La aplicación va a usar la misma lógica de negocio y backend tanto en entorno web como en las aplicaciones móviles. El flujo, cumpliendo con los requisitos funcionales, será:

1. Registro/Inicio de sesión por parte del usuario
2. CRUD de vehículos registrados a nombre del usuario
3. Dentro de la ficha de cada vehículo:
 - a. Acceder a los datos completos de vehículos
 - b. Historial completo de intervenciones realizadas en orden cronológico inverso (más recientes primero)
4. CRUD de intervenciones por cada vehículo.

El diseño genérico permite registrar gran variedad de vehículos (coches, motos, furgonetas, camiones, vehículos de movilidad personal, etc...), lo que hace la aplicación adecuada para cualquier tipo de usuario, sin complicar posibles ampliaciones futuras.

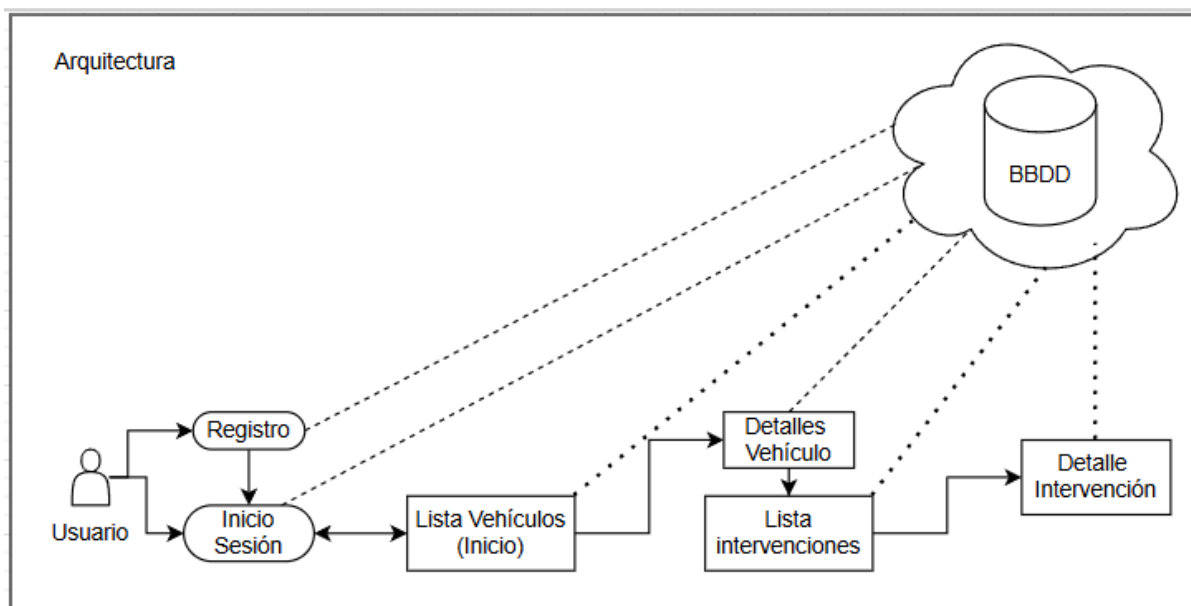
También se añadirán medidas de seguridad básica para el acceso, permitiendo la persistencia de datos (Base de Datos SQL), UX/UI claro, y documentación completa del sistema.

A continuación se detallan diferentes esquemas/diagramas para complementar esta descripción.

Arquitectura De La Solución

Figura 1

Esquema Arquitectura de la Solución.



En la Figura 1 se detalla la estructura de la aplicación, para que se vea cómo se comunican las partes. Se trata de una arquitectura cliente-servidor, donde la app Flutter es el cliente y Supabase actúa como servidor (backend) en la nube, gracias a esto se puede separar la lógica de negocio de la gestión de los datos, lo cual facilita tanto el desarrollo como el mantenimiento y la escalabilidad de todo el sistema.

La explicación técnica se podría resumir en que el usuario interactúa con la aplicación (Flutter), ésta lanza la petición mediante el API Rest de Supabase, que procesa la solicitud y accede a la base de datos para responder al cliente y que la interfaz actualice automáticamente lo que ve el usuario.

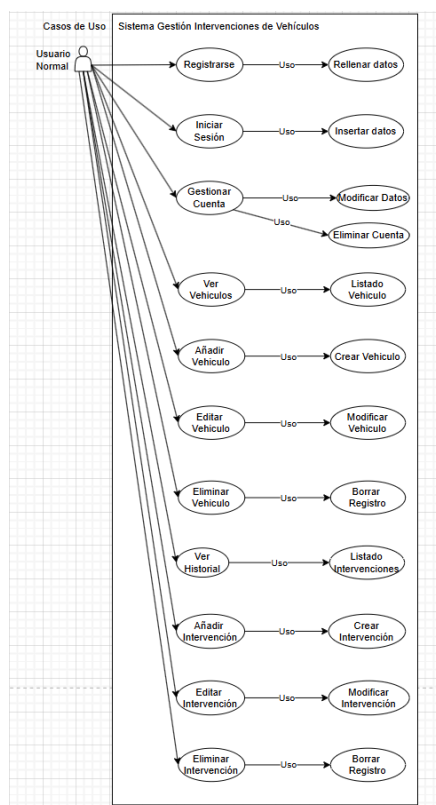
Casos De Uso

La aplicación permite a los usuarios registrarse, iniciar sesión y gestionar su perfil, además también pueden añadir, editar y eliminar sus vehículos, gestionar las intervenciones

realizadas en éstos y consultar el historial de intervenciones. El usuario con rol de administrador no tiene limitadas estas funciones sólo a sus vehículos. En la Figura 2 se puede ver un esquema de estos casos de uso y a continuación una tabla por cada uno donde se mencionan los aspectos más técnicos.

Figura 2

Esquema General Casos de Uso



PROMETEO

Tabla 2
Caso de uso Registrarse

DESCRIPCIÓN: Permite a un usuario crear una cuenta en la aplicación proporcionando sus datos básicos	
PRECONDICIONES: El usuario NO debe estar registrado previamente.	POSTCONDICIONES: Se crea un nuevo usuario en la base de datos y puede iniciar sesión.
DATOS ENTRADA Nombre Email Contraseña.	DATOS SALIDA Confirmación de registro O mensaje de error.
TABLAS: USUARIO	CLASES: UsuarioService AuthService (Supabase)
INTERFACES: PantallaRegistro, APIAuth	

Tabla 3
Caso de uso Iniciar Sesión

DESCRIPCIÓN: Permite al usuario acceder a la aplicación mediante sus credenciales.	
PRECONDICIONES: El usuario debe estar registrado.	POSTCONDICIONES: Se inicia sesión y se obtiene acceso a la aplicación.
DATOS ENTRADA Email Contraseña.	DATOS SALIDA Token de sesión O error de autenticación.
TABLAS: USUARIO	CLASES: AuthService (Supabase)
INTERFACES: PantallaLogin, APIAuth	

Tabla 4
Caso de uso Gestionar Cuenta

DESCRIPCIÓN: Permite al usuario modificar sus datos personales o eliminar su cuenta.	
PRECONDICIONES: Usuario autenticado.	POSTCONDICIONES: Datos actualizados o cuenta eliminada.
DATOS ENTRADA Nombre Email Contraseña	DATOS SALIDA Confirmación de operación

PROMETEO

TABLAS: USUARIO	CLASES: UsuarioService
INTERFACES: PantallaPerfil, APIUsuario	

Tabla 5

Caso de uso Ver Vehículo

DESCRIPCIÓN: Muestra la lista de vehículos del usuario.	
PRECONDICIONES: Usuario autenticado.	POSTCONDICIONES: Listado mostrado
DATOS ENTRADA ID del usuario.	DATOS SALIDA Lista de vehículos
TABLAS: VEHICULOS	CLASES: VehiculoService
INTERFACES: PantallaListaVehiculos, APIVehiculo	

Tabla 6

Caso de uso Añadir Vehículo

DESCRIPCIÓN: Permite al usuario registrar un nuevo vehículo.	
PRECONDICIONES: Usuario autenticado.	POSTCONDICIONES: El vehículo queda almacenado en la base de datos.
DATOS ENTRADA Tipo, Marca Modelo Matrícula Año Bastidor Km	DATOS SALIDA Confirmación de creación.
TABLAS: VEHICULOS	CLASES: VehiculoService
INTERFACES: PantallaNuevoVehiculo, APIVehiculo	

Tabla 7*Caso de uso Editar Vehículo*

DESCRIPCIÓN: Permite modificar los datos de un vehículo existente.	
PRECONDICIONES: Usuario autenticado y propietario del vehículo.	POSTCONDICIONES: Datos actualizados.
DATOS ENTRADA Campos del vehículo modificados.	DATOS SALIDA Confirmación de actualización.
TABLAS: VEHICULOS	CLASES: VehiculoService
INTERFACES: PantallaEditarVehiculo, APIVehiculo	

Tabla 8*Caso de uso Eliminar Vehículo*

DESCRIPCIÓN: Permite eliminar un vehículo registrado.	
PRECONDICIONES: Usuario autenticado y propietario del vehículo.	POSTCONDICIONES: El vehículo y sus intervenciones pueden eliminarse.
DATOS ENTRADA ID del vehículo.	DATOS SALIDA Confirmación de eliminación.
TABLAS: VEHICULOS INTERVENCION	CLASES: VehiculoService
INTERFACES: APIVehiculo	

Tabla 9*Caso de uso Ver Historial*

DESCRIPCIÓN: Muestra el historial de intervenciones de un vehículo ordenado por fecha.	
PRECONDICIONES: Usuario autenticado y vehículo existente.	POSTCONDICIONES: Listado mostrado
DATOS ENTRADA ID del vehículo	DATOS SALIDA Lista de intervenciones.
TABLAS: INTERVENCION	CLASES: IntervencionService
INTERFACES: PantallaHistorial, APIIntervencion	

Tabla 10*Caso de uso Registrar Intervención*

DESCRIPCIÓN: Permite registrar una intervención en un vehículo.	
PRECONDICIONES: Usuario autenticado y vehículo existente.	POSTCONDICIONES: Intervención registrada.
DATOS ENTRADA Tipo Descripción Coste Km Fecha Adjunto	DATOS SALIDA Confirmación de creación.
TABLAS: INTERVENCION	CLASES: IntervencionService
INTERFACES: PantallaNuevaIntervencion, APIIntervencion	

Tabla 11*Caso de uso Modificar Intervención*

DESCRIPCIÓN: Permite modificar una intervención existente.	
PRECONDICIONES: Usuario autenticado.	POSTCONDICIONES: Datos actualizados.
DATOS ENTRADA Campos de intervención modificados.	DATOS SALIDA Confirmación de actualización.
TABLAS: INTERVENCION	CLASES: IntervencionService
INTERFACES: PantallaEditarIntervencion, APIIntervencion	

Tabla 12*Caso de uso Eliminar Intervención*

DESCRIPCIÓN: Permite eliminar una intervención.	
PRECONDICIONES: Usuario autenticado.	POSTCONDICIONES: Intervención eliminada
DATOS ENTRADA ID intervención	DATOS SALIDA Confirmación de eliminación.
TABLAS: INTERVENCION	CLASES: IntervencionService
INTERFACES: APIIntervencion	

En resumen, una vez el usuario se registra, puede iniciar sesión y gestionar su perfil (modificar datos o eliminar la cuenta). Dentro de la aplicación puede gestionar sus vehículos y las intervenciones asociadas a cada uno de ellos (para ambos casos añadir, editar, eliminar o consultar).

Diseños

Diagrama De Clases

La estructura lógica del sistema se basa en las 3 tablas que aparecen en la base de datos (ver figuras 4 y 5 para el esquema entidad relación y esquema de la base de datos), a la cual se le añaden dos clases más que vienen por defecto gracias al uso de Supabase, por un lado el servicio de autenticación (AuthService), que se relaciona directamente con la clase usuario y los métodos que implementa, y por otro lado el SupabaseService, que implemente los métodos para añadir, editar, actualizar y eliminar vehículos e intervenciones.

Figura 3

Diagrama De Clases

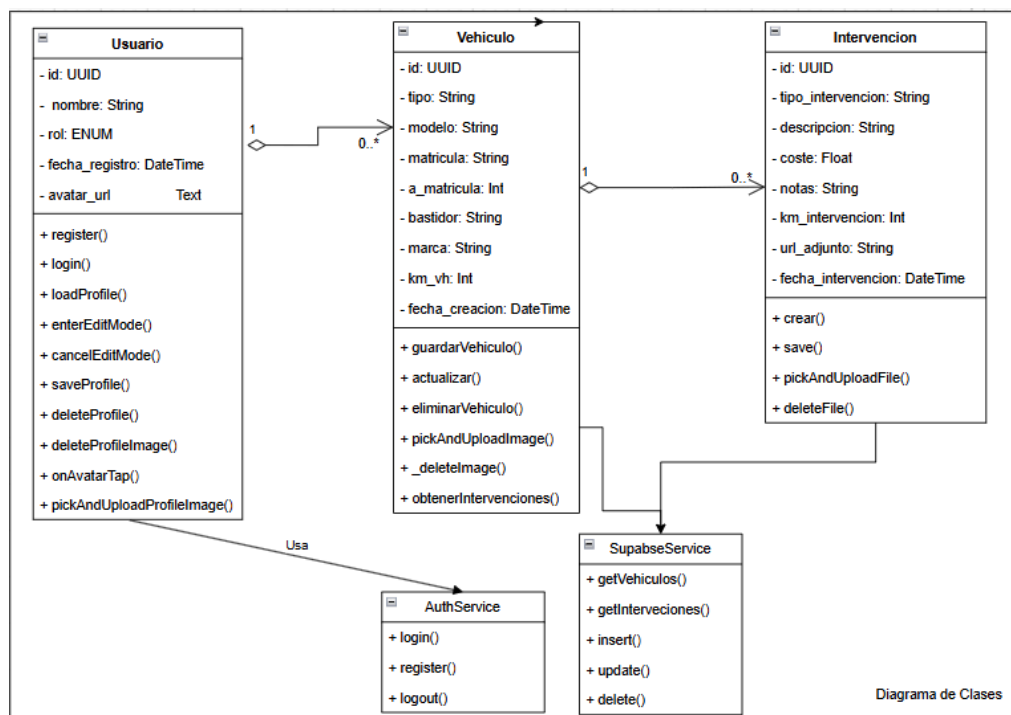


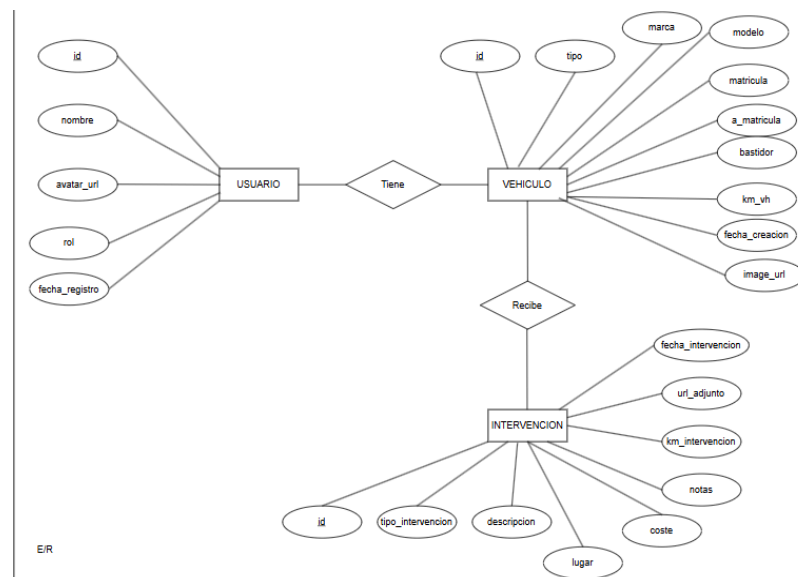
Diagrama de Clases

Se usa una programación orientada a objetos (POO). Los campos de cada clase se explican más adelante (Figuras 4 y 5) y como factor diferenciador y clave de este diagrama, se ven los métodos, para los de las clases creadas manualmente se ha usado indistintamente el castellano o el inglés ya que los métodos ya son bastante autoexplicativos. Para los métodos propios de Supabase, se han usado los nombres por defecto, en AuthService: login() que se relaciona con el del mismo nombre de Usuario, register() que se relaciona con registrar() en Usuario y se añade el logout() para cerrar sesión. En SupabaseService tenemos 2 getters, para obtener el listado de vehículos e intervenciones respectivamente, y por último tenemos los métodos restantes para completar el CRUD: insert(), update() y delete().

Diagrama Entidad Relación

Figura 4

Diagrama Entidad-Relación



Este diagrama E-R, representa el modelo conceptual sobre el que se ha trabajado para crear la BBDD. Identificando las 3 entidades principales (Usuario, Vehículo e Intervención), y los atributos relacionados a cada una de ellas, permitiendo definir claramente la estructura que va a tener la información en el sistema, antes de implementar

PROMETEO

la base de datos. Ahora vamos a detallar cada una de las entidades:

Entidad Usuario, para representar a quienes usan la aplicación, cada uno tiene su cuenta individual para acceder al sistema y gestionar sus vehículos, siendo una entidad fundamental ya que es el punto de partida en la consulta de los vehículos y su información asociada.

Entidad Vehículo, para representar los vehículos que registran los usuarios, para tener un historial de intervenciones. Cada vehículo solo pertenece a un solo usuario, por lo que cada usuario solo puede gestionar los vehículos que le pertenecen.

Entidad Intervención, representa los mantenimientos, reparaciones, mejoras, etc... que se realizan sobre un vehículo, estando cada una de ellas asociada a un vehículo en particular.

El detalle de los campos de cada tabla/entidad se puede consultar en la Figura 4.

Diagrama De La Base De Datos

La base de datos se ha estructurado de forma relacional de la manera más simple posible (Ver Figura 5), con tres entidades principales, mostrando en el diagrama la relación entre los usuarios y sus vehículos, y de éstos con las intervenciones asociadas. Además Supabase genera de manera automática una cuarta tabla (auth.users.id), que registra internamente algunos datos de los usuarios (id, que coincide con la tabla usuarios, email, fecha de registro, último inicio de sesión, y algunos otros campos que no son relevantes a priori para este proyecto).

Al trabajar en el entorno Supabase, se han tenido que crear una serie de campos ENUM: rol_usuario para la tabla usuarios que distingue entre admin y usuario (standard), y en la tabla intervenciones, se han creado dos, uno para el tipo_intervencion (mejora, reparación, modificación, revisión, etc...) y otro para lugar_intervencion (casa o taller).

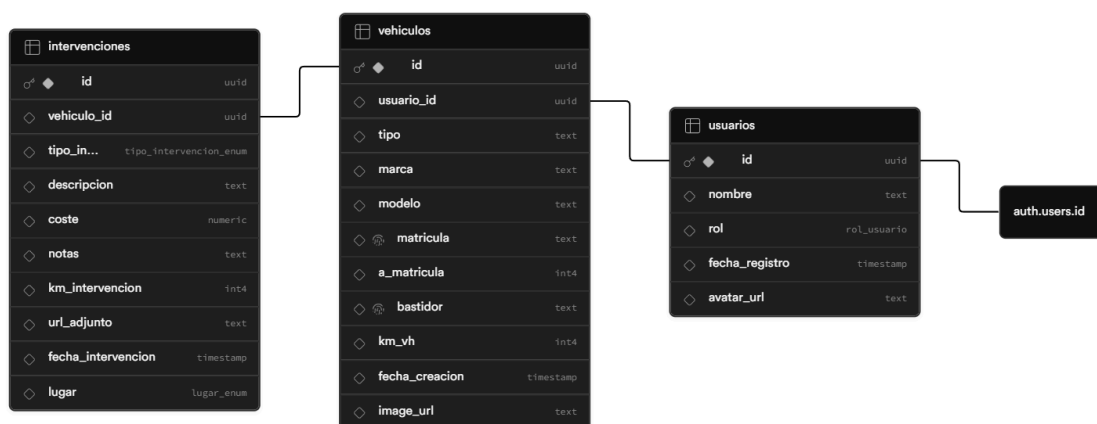
Obviando este detalle distintivo de la forma de trabajar con Supabase, la prioridad del modelo, como en todo el proyecto, es la simplicidad, para facilitar la implementación y la

PROMETEO

escalabilidad futura, por eso hemos aprovechado las tablas que genera Supabase y la hemos complementado con una propia de usuarios para tener un mejor control de los aspectos que nos interesan para este proyecto.

Figura 5

Diagrama con detalle de los campos de la base de datos



Al usar Supabase por defecto se usa como Sistema Gestor de Bases de Datos (SGBD) PostgreSQL; ampliamente utilizado hoy en día gracias a su robustez y rendimiento, además de soportar datos avanzados y mantener la integridad referencial.

En todas las tablas tenemos un campo `id`, que actúa como clave primaria (PK), identificando cada registro del sistema (ya sea usuario, vehículo o intervención) inequívocamente, mediante el tipo UUID (Universally Unique Identifier), que se genera de forma aleatoria gracias a la función `gen_random_uuid()`. Cada tabla tiene una serie de campos bastante autoexplicativos con su nombre.

Para completar la tabla autogenerada en Supabase, `auth.users.id`, se ha creado la tabla `usuarios`, así se garantiza que un usuario quede registrado correctamente y se recojan los datos útiles para este proyecto, 'nombre' (de usuario) y un 'email', que son campos tipo

PROMETEO

TEXT y con la restricción de no poder ser nulos (NOT NULL) y para evitar duplicidades de usuario se añade otra restricción UNIQUE. A nivel de sistema se almacena el 'rol' (por defecto con el valor 'usuario') que es un ENUM y la 'fecha_registro', siendo por defecto NOW(), del tipo TIMESTAMP, también damos la opción de añadir una imagen de perfil. Dejando campos sensibles como la contraseña para la tabla auth.users.id de Supabase.

A la hora de registrar vehículos, tenemos que relacionarlos con 'usuario_id', que actúa como clave foránea (FK) en esta tabla, y necesitamos saber qué 'tipo' de vehículo es, su 'marca', 'modelo y los kilómetros recorridos ('km_vh'), e identificarlo por su 'matrícula', año de matriculación ('a_matricula'), por su 'bastidor', siendo todos ellos campos tipo TEXT, dando también la opción de guardar una imagen del vehículo, dejando a nivel de sistema la 'fecha_creación', que es tipo TIMESTAMP. Se implementa la restricción UNIQUE a 'matricula' y 'bastidor'.

Hablando de las Intervenciones, además de relacionarlas con 'vehiculo_id' como FK de la tabla Vehiculos, tenemos que seleccionar el ENUM 'tipo_intervención' realizada, la 'descripcion' para detallar que se ha hecho, podemos añadir 'notas', guardar adjuntos en Supabase ('url_adjunto'), y elegir el 'lugar_intervencion' que es ENUM. Por último podemos añadir el 'coste' (tipo NUMERIC para trabajar con decimales), así como la fecha de la intervención (TIMESTAMP).

Las relaciones entre las tablas de la base de datos son: Usuarios → Vehiculos 1:N, es decir un usuario puede registrar muchos vehículos, y Vehiculos → Intervenciones 1:N, lo que quiere decir que un vehículo puede tener registradas múltiples intervenciones. Y entre Usuarios y auth.users.id la relación es 1:1.

A estas relaciones, mediante las FK se ha añadido la restricción ON DELETE CASCADE, para que si se borra un usuario de la base de datos, se borren todos sus vehículos y si se borra un vehículo, desaparezcan también todas las intervenciones.

Seguridad Y Control De Acceso: Row Level Security

Al hablar de Row Level Security (RLS), hablamos de un mecanismo de seguridad de PostgreSQL que permite definir políticas de acceso a nivel de fila, estas políticas, a diferencia de las que se aplican a nivel tabla, controlan qué datos puede ver o modificar cada usuario en base a su identidad, garantizando que no puedan acceder a datos que no le pertenecen, aunque tuviese acceso a nivel técnico a la base de datos, llegado el caso.

En este proyecto, Supabase gestiona la parte de autenticación y mediante la función `auth.uid()` devuelve el identificador único del usuario logueado, y es de aquí de donde cuelgan todas las políticas RLS, ya que compara la identidad del usuario para saber de qué registros es propietario

Políticas Aplicadas Por Tabla

En la tabla Usuarios se almacena el perfil de cada usuario (id, nombre, avatar, etc...), el campo id coincide con la tabla nativa de Supabase Auth, donde se almacenan el resto de datos más sensibles de cada usuario. Al coincidir id con `auth.id` se garantiza que cada usuario solo accede a los datos de su perfil (solo a las filas relacionadas con su id en las tablas).

Tabla 13

Políticas RLS tabla Usuarios

Política	Operación	Condición	Expresión SQL
Usuario puede ver su perfil	SELECT	USING	<code>auth.uid() = id</code>
Usuario puede insertarse	INSERT	WITH CHECK	<code>auth.uid() = id</code>
Usuario actualiza su perfil	UPDATE	USING	<code>auth.uid() = id</code>
Usuario puede borrar su perfil	DELETE	USING	<code>auth.uid() = id</code>

PROMETEO

En la tabla Usuarios se almacena el perfil de cada usuario (id,nombre, avatar, etc...), el campo id coincide con la tabla nativa de Supabase Auth, donde se almacenan el resto de datos más sensibles de cada usuario. Al coincidir id con auth.id se garantiza que cada usuario solo accede a los datos de su perfil (solo a las filas relacionadas con su id en las tablas).

Tabla 14

Políticas RLS tabla Vehículos

Política	Operación	Condición	Expresión SQL
Ver vehículos propios	SELECT	USING	usuario_id = auth.uid()
Insertar vehículos propios	INSERT	WITH CHECK	auth.id() = usuario_id
Actualizar vehículos propios	UPDATE	USING	auth.id() = usuario_id
Eliminar vehiculos propios	DELETE	USING	auth.id() = usuario_id

La tabla intervenciones no contiene directamente identificadores de usuario, si no que se relaciona con la tabla vehículo a través de la FK vehiculo_id. Todas las políticas de esta tabla usan la subconsulta EXISTS para verificar que el vehículo al que pertenece la intervención pertenece al usuario, esta forma de verificación indirecta garantiza incluso la seguridad entre las relaciones.

Tabla 15*Políticas RLS tabla Intervenciones*

Política	Operación	Condición	Expresión SQL
Ver intervenciones propias	SELECT	USING	EXISTS (SELECT 1 FROM vehiculos v WHERE v.id = vehiculo_id AND v.usuario_id = auth.uid())
Insertar intervenciones propias	INSERT	WITH CHECK	EXISTS (SELECT 1 FROM vehiculos v WHERE v.id = vehiculo_id AND v.usuario_id = auth.uid())
Actualizar Intervenciones propias	UPDATE	USING	EXISTS (SELECT 1 FROM vehiculos v WHERE v.id = vehiculo_id AND v.usuario_id = auth.uid())
Eliminar intervenciones propias	DELETE	USING	EXISTS (SELECT 1 FROM vehiculos v WHERE v.id = vehiculo_id AND v.usuario_id = auth.uid())

Políticas De Storage

Se han creado también políticas de acceso sobre el bucket de almacenamiento de los archivos ('archive'), donde se guardan tanto las fotos de perfil de los usuarios como las fotos de los vehículos y documentos relacionados con las intervenciones. Al ser un proyecto académico y estar con la capa gratuita de Supabase se ha limitado (e implementado a nivel de código) un tope de 2MB por archivo (tanto imágenes como PDF), para comprobar el correcto funcionamiento y no saturar la capacidad total otorgada por Supabase.

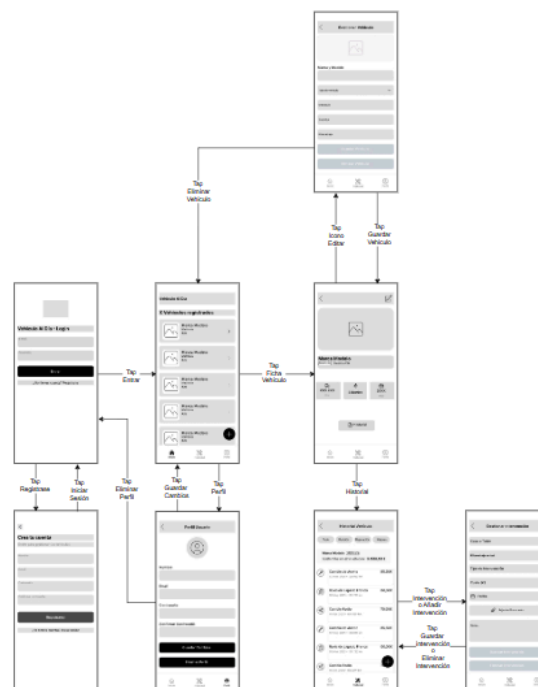
Impacto En La Arquitectura De La Aplicación

Haber implementado tanto las políticas RLS como de Storage ha influido directamente en cómo se ha diseñado la app y en concreto la capa de datos. Los repositorios en Flutter no necesitan de filtros manuales por id de usuario ya que de eso se encarga Supabase directamente antes de devolver los datos. Esto ha ayudado a simplificar el código, eliminando también errores humanos que podrían exponer datos de otros usuarios.

Diagrama De Flujo De Navegación

Figura 6

Flujo Básico De Navegación



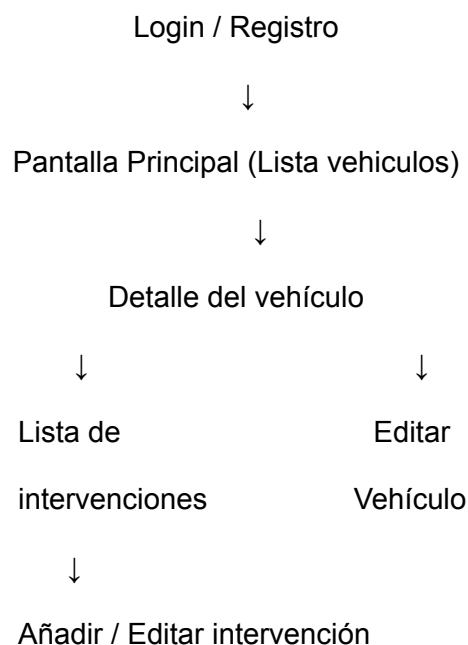
La base de este diagrama son los wireframes, que nos han permitido centrarnos en la estructura y organización de los elementos por cada pantalla, obviando aspectos visuales, como los colores o estilos, con ellos antes del diagrama de flujo (Figura 6), hemos validado la distribución y jerarquización de los elementos, antes de pasar al diagrama de flujo en sí. En los wireframes, además de las pantallas del diagrama hemos incluido estados vacíos de la pantalla principal y la pantalla del historial.

PROMETEO

Con el diagrama anterior podemos observar de forma resumida como se puede navegar por la app, ya que vemos como las diferentes pantallas se conectan entre sí, para tener una idea visual del recorrido que realiza el usuario dentro de la aplicación desde que se registra para acceder al sistema hasta que usa las diferentes funcionalidades del día a día con la aplicación. Así podemos garantizar que la experiencia general y con la navegación en particular, sea clara, sencilla e intuitiva.

Los wireframes que ilustran el diagrama de flujo, representan las pantallas de forma esquemática y se han diseñado en Figma. Siendo la base para el posterior desarrollo de las interfaces. El objetivo principal ha sido el de minimizar el número de pantallas de cara a la realización de las diferentes operaciones, permitiendo un rápido acceso a la información más relevante y que se busca en cada momento.

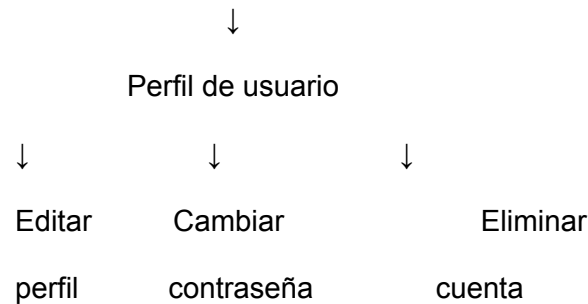
La estructura general de navegación, se realiza de forma jerarquizada, siendo la pantalla principal el punto central desde el que acceder al resto de funcionalidades. El flujo principal puede resumirse de la siguiente manera:



Paralelamente hay otro flujo desde la pantalla principal para que el usuario pueda gestionar su perfil:

PROMETEO

Pantalla Principal (Lista de Vehículos)



Detallamos a continuación cada una de las pantallas básicas de las que parte el proyecto:

- Pantalla de Inicio de Sesión: Es la primera pantalla que se muestra al abrir la app. Permite a los usuarios autenticarse en el sistema, introduciendo sus credenciales (mail, y contraseña), y pulsando el botón correspondiente, si todo es correcto, redirige a la pantalla principal de la aplicación. Antes de pulsar nada, también redirige a la pantalla de registro por si el usuario aún no está registrado.

Figura 7

Wireframe Pantalla de Inicio de Sesión



- Pantalla de Registro: Permite al usuario crear una cuenta introduciendo los datos solicitados (nombre, mail, contraseña), una vez registrado se podrá iniciar sesión y acceder a todas las funcionalidades.

Figura 8*Wireframe Pantalla de Registro*

- Pantalla Principal: En formato Dashboard, muestra una lista de los vehículos registrados por el usuario que ha iniciado sesión. Es la pantalla central de la aplicación, desde aquí se pueden realizar la mayoría de las acciones (visualizar la lista de vehículos, acceder al detalle de cada vehículo, añadir nuevo vehículo y acceder a la pantalla de perfil), si no tiene vehículos registrados la pantalla muestra un estado vacío, invitando al usuario a registrar el primero.

Figura 9*Wireframe Pantalla Principal*

- Pantalla Detalle Vehículo: Muestra la información completa del vehículo seleccionado (marca, modelo, matrícula, bastidor, año matriculación, kilometraje actual) y permite realizar diferentes acciones (editar o eliminar el vehículo, y acceder al historial de intervenciones).

Figura 10*Wireframe Pantalla Detalle Vehículo*

- Pantalla Lista de Intervenciones: Muestra el historial de intervenciones registradas en la aplicación, cada intervención es una tarjeta que muestra el tipo, la fecha el kilometraje y el coste de la intervención. Desde esta pantalla además de consultar el historial completo, se pueden añadir nuevas intervenciones o entrar en el detalle de cada una. Si no tiene intervenciones registradas se muestra un estado vacío, que invita al usuario a registrar la primera.

Figura 11*Wireframe Pantalla Historial Vehículo*

- Pantalla Gestión de Intervenciones: Permite al usuario gestionar las intervenciones existentes (crear, editar o eliminar). Se puede trabajar con los siguientes campos: Tipo de intervención, descripción (para ver si se ha hecho en casa o en taller), coste, kilometraje, notas y archivos adjuntos.

Figura 12*Wireframe Pantalla Gestionar Intervenciones*

- Pantalla Perfil de Usuario: Permite gestionar al usuario la información dentro de la aplicación: Consultar y/o editar sus datos, cambiar de contraseña o eliminar la cuenta. En caso de que suceda esto último se borrarán tanto los vehículos como las intervenciones asociadas a éstos.

Figura 13*Wireframe Pantalla Perfil de Usuario*

El diseño se ha planteado con el objetivo de que el usuario pueda tener una experiencia lo más sencilla e intuitiva posible. La pantalla principal muestra la información más relevante para el usuario (el listado de vehículos registrados), reduciendo los pasos para acceder a la información. Además la estructura jerarquizada permite que los usuarios consulten el historial en cuanto accedan a la ficha del vehículo. Esto da una interfaz limpia,

PROMETEO

intuitiva y fácil de usar, reduciendo al mínimo la complejidad de la aplicación, por tanto mejorando la experiencia de usuario (UX).

Interfaces

A partir de los wireframes usados en el diagrama de flujo, se ha desarrollado el diseño visual completo de la aplicación, incorporando la paleta de colores, tipografías y estilos previamente definidos. El objetivo era tener una interfaz con claridad visual, información bien jerarquizada, tamaño adecuado de botones y espacio suficiente entre elementos.

Para el desarrollo de la interfaz de usuario se ha seguido el enfoque Mobile First, diseñando inicialmente las pantallas para dispositivos móviles, que posteriormente se adaptaran a dispositivos más grandes (tablets o navegadores web de escritorio). Como dispositivo base se ha usado una resolución de pantalla equivalente a un iPhone 13/14 (390x844px), para trabajar desde los móviles más pequeños. Este enfoque (Mobile First) prioriza la usabilidad en pantallas reducidas, facilitando la creación de interfaces más simples y claras, mejorando también la adaptabilidad a resoluciones variadas y permitiendo el posterior escalado a dispositivos con pantallas más grandes (ya sean otros móviles, tablets o dispositivos de escritorio). Se ha tomado este planteamiento con la idea de facilitar la consulta y actualización de las intervenciones, una vez terminadas de realizar.

Antes de definir lo anterior, así como el resto de variables sobre el diseño definitivo, se ha realizado una pequeña investigación de mercado con una encuesta digital dirigida a potenciales usuarios de la aplicación. Con ella se ha obtenido feedback (Unas 100 encuestas válidas) sobre las preferencias en cuanto a: Nombre comercial de la aplicación, paleta de colores, preferencias sobre las pantallas y cómo organizarlas. Las tendencias detectadas en las encuestas han sido:

- El 49'5% de los encuestados se decantó por el nombre VehiculoAIDia (o su derivado con espacios)

- El 57% de los que respondieron seleccionó esta paleta de colores



Tabla 16

Colores Identidad de Marca

Elemento	Color	Código
Color primario	Azul corporativo	#0047AB
Color secundario	Gris acero	#708090
Color de fondo	Blanco nube	#F4F7F6
Color de acción	Naranja alerta	#FF8C00
Texto principal	Negro carbón	#1A1A1A
Texto secundario	Gris profundo	#4A4A4A

- Casi el 55% de los potenciales usuarios, indicaron que consideran más útil ver la lista de vehículos registrados nada más entrar a la aplicación

Gracias a estos resultados se ha realizado la aplicación de manera acorde, adaptando la idea primigenia a las respuestas de los encuestados.

Toda la parte del diseño de interfaces (igual que los bocetos, wireframes, etc...) se ha realizado en Figma, ya que es una herramienta ampliamente utilizada para estos menesteres. Además de los diseños propiamente dichos hemos usado las funcionalidades de creación de componentes reutilizables (botones, tarjetas, etc...), y los estilos de color y tipografía creando las bibliotecas correspondientes, por ultimo también nos ayudará a crear un prototipo interactivo de la aplicación.

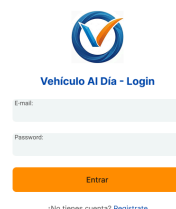
En cuanto a tipografía se ha optado por dos opciones principalmente: La principal es Inter, usada para títulos y elementos de navegación y Roboto para información sobre datos, contenido técnico, campos de formularios o intervenciones. Y en ambas se ha usado una jerarquía tipográfica que permita diferenciar los niveles de información:

Tabla 17*Biblioteca tipográfica de la aplicación.*

Elemento	Tamaño	Estilo
Títulos principales	22 px	Bold
Subtítulos	18 px	SemiBold
Texto de contenido	15 px	Regular
Botones	16 px	Medium

Se han usado también componentes reutilizables (botones de acción, tarjetas varias, campos, iconos) para mantener la coherencia visual entre pantallas, así como una identidad visual consistente y definida en toda la aplicación.

Teniendo ya todo esto hecho, se realiza un prototipo interactivo para simular la navegación entre las distintas pantallas, para comprobar, antes de la programación el comportamiento visual de la interfaz, el flujo de navegación y ver posibles problemas de usabilidad, pudiendo además realizar muestras del funcionamiento sin tener el producto final terminado. Una vez explicado todo lo general aplicable a la aplicación, se procede a explicar cada pantalla en detalle:

Figura 14*Pantalla de Inicio de sesión*

Aquí la interfaz se compone del logo y título de la aplicación en la parte superior, justo debajo se encuentran los campos de correo electrónico y contraseña, para que los

PROMETEO

usuarios introduzcan sus credenciales. Y por último encontramos el botón de inicio de sesión, que valida con el sistema las credenciales introducidas por el usuario, junto con un enlace a la pantalla de registro en caso de que el usuario todavía no tenga cuenta.

Cuando el usuario introduce sus credenciales y pulsa el botón el servicio AuthService de Supabase recibe la solicitud, si son válidas redirige a la pantalla principal y en caso contrario muestra mensaje de error.

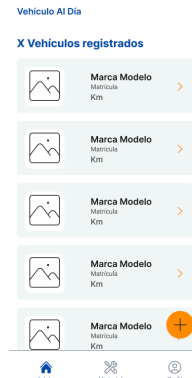
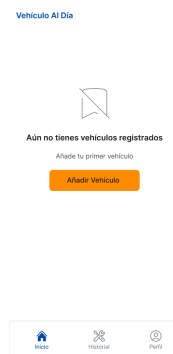
Figura 15

Interfaz de Registro

La imagen muestra la interfaz de registro de la aplicación. En la parte superior izquierda hay un icono de flecha hacia atrás. Debajo de él, el título "Crea tu cuenta" en azul, seguido de un subtítulo "Únete para gestionar tus vehículos" en gris. Hay cuatro campos de entrada de texto: "Nombre", "Email", "Contraseña" y "Confirma contraseña". Debajo de estos campos hay un botón naranja con el texto "Registrarse". En la parte inferior, hay un enlace que dice "¿Ya tienes cuenta? Inicia sesión".

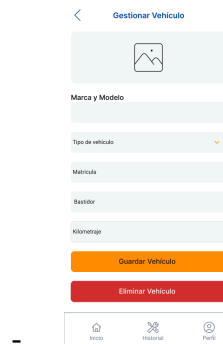
Esta pantalla permite a los usuarios crear una cuenta, proceso necesario para disfrutar de las funcionalidades de la aplicación. Se incluyen todos los campos a rellenar (nombre de usuario, correo electrónico y contraseña) y el botón para formalizar el registro.

Una vez completado el registro se envían los datos a Supabase mediante el método register() del servicio AuthService, si se completa correctamente, se crea el usuario (en la tabla correspondiente) y se le permite iniciar sesión.

Figura 16*Interfaz Principal (Dashboard/Lista de vehículos)***Figura 17***Interfaz Principal (Dashboard/Lista de vehículos) vacía*

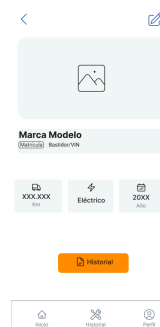
Aquí se muestra el listado de vehículos que el usuario ha registrado en la aplicación, es el centro de navegación de todo el sistema. En la parte superior se muestra el nombre de la aplicación, justo debajo la lista de vehículos, en formato tarjeta (contiene: marca, modelo, matrícula y kilometraje), y un FAB con un símbolo + para añadir más vehículos.

Al cargar la pantalla SupabaseService mediante el método `getVehiculos()` carga los datos de la pantalla, y en caso de que el usuario no tenga vehículos asociados se muestra un estado vacío invitando a registrar el primero.

Figura 18*Interfaz Gestionar vehículo básica*


Esta pantalla permite al usuario registrar un nuevo vehículo, rellenando: tipo de vehículo, marca, modelo, matrícula, año matriculación, bastidor y kilometraje.

Cuando el usuario completa el formulario y pulsa el botón guardar, mediante el método insert() se envían los datos usando SupabaseService a la BBDD, concretamente a la tabla vehículos.

Figura 19*Interfaz Detalle de Vehículo básica*


Aquí se muestra la información completa del vehículo seleccionado: Marca, modelo, matrícula, año matriculación, bastidor y kilometraje, pudiendo editar o eliminar el vehículo y acceder al historial de intervenciones. Todos estos datos se recuperan de nuevo con una consulta a SupabaseService.

Figura 20*Interfaz Historial (Lista de intervenciones)***Figura 21***Interfaz Historial (Lista de intervenciones) vacía*

Esta pantalla muestra el historial completo de intervenciones registradas en la aplicación. Cada intervención muestra el tipo, la fecha, kilometraje y coste.

El usuario puede acceder a los detalles de cada intervención.

Figura 22*Interfaz de gestión de intervenciones básica*

Se permite registrar nuevas intervenciones, rellenando los campos correspondientes (tipo, donde se ha hecho, coste, kilometraje, notas, adjuntos y fecha).

PROMETEO

Una vez introducidos los datos, mediante el método insert() de SupabaseService se guardan en la base de datos, tabla de intervenciones.

Figura 23

Interfaz Perfil de usuario básica

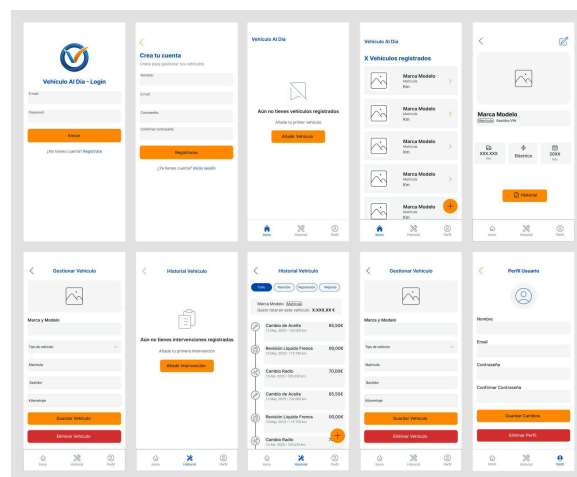


Esta pantalla permite que se realicen todas las gestiones sobre la información del usuario, consulta y edición de datos, cambios de contraseña y eliminación de cuenta.

Las modificaciones se envían al backend mediante los servicios de autenticación y acceso de datos que proporciona el backend de Supabase.

Figura 24

Collage con todas las interfaces básicas



Prototipo interactivo

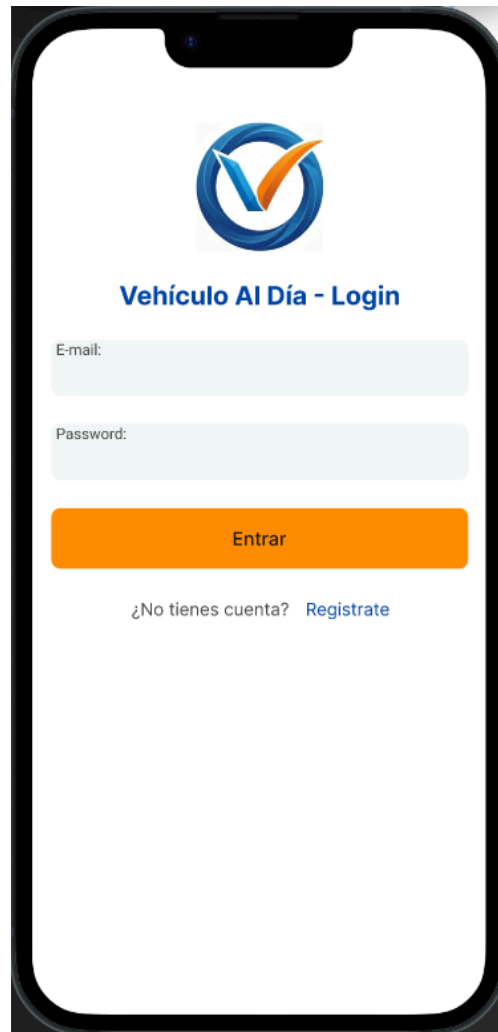
Además del diseño estático de las interfaces, se ha preparado un prototipo básico usando Figma, para simular una pequeña navegación entre las diferentes pantallas, para así poder validar el flujo completo del usuario antes del desarrollo. En este prototipo se puede:

- Navegar entre pantallas

- Simular interacciones básicas
- Validar UX (Experiencia de Usuario)

Figura 25

Captura de pantalla del Prototipo básico



El prototipo interactivo básico puede consultarse [aquí](#).

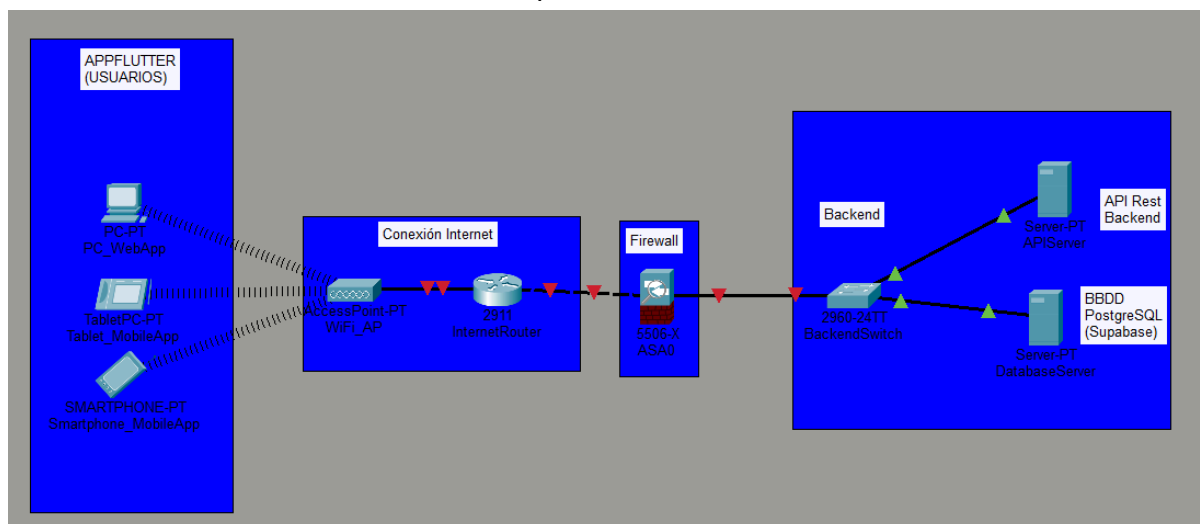
Enlace al proyecto base (Wireframes, interfaces, navegación...), se puede visitar [aquí](#).

Las interfaces finales se han modificado en función de las necesidades que se han ido detectando según se desarrolla el proyecto, añadiendo snackbars, ventanas flotantes, pantallas condicionales (cambios de título según la actividad que se esté realizando, etc...) etc...

Diagrama De Red

Figura 26

Estructura red del funcionamiento de la aplicación.



El sistema de la aplicación usa la arquitectura cliente-servidor, donde los diferentes dispositivos acceden a la aplicación, como se muestra en la figura realizada con Cisco Packet Tracer, modelando una simulación de los elementos de red que intervienen y cómo se comunican entre ellos.

Al ser una aplicación multiplataforma se han representado diferentes dispositivos clientes, que se conectan al Access Point mediante WiFi (aunque el PC podría también hacerlo por cable), que da conectividad a internet conectando al router.

El router es el punto central de comunicación entre los dispositivos de la infraestructura, siendo el gateway (puerta de enlace), y gestionando el tráfico, que llega al switch encargado de distribuir el tráfico a la parte del servidor que corresponda. En la parte de servidores se han definido dos componentes principales:

Se distinguen dos tipos de servidores, el APIServer, donde se encuentra toda la lógica de negocio, procesamiento de peticiones, etc... que ofrece los servicios de la aplicación mediante API REST.

El de la base de datos, simula toda la infraestructura de Supabase donde se almacenan los datos de la aplicación (usuarios, vehículos e intervenciones).

PROMETEO

Esto hace que los clientes no acceden directamente a la base de datos, sino que envían las peticiones a la API, que gestionando la lógica de negocio realiza las queries (consultas) a la BBDD. Esta forma de trabajo mejora la seguridad, escalabilidad y mantenimiento del sistema al tener separación entre las capas.

Tecnología

Las tecnologías seleccionadas permiten implementar una arquitectura moderna basada en aplicaciones cliente conectadas a la nube, lo que facilita la escalabilidad, mantenimiento y acceso multiplataforma. Además al ser un proyecto de un solo programador se ha optado por la simplificación y la máxima ayuda de las herramientas de las que dispone. Las tecnologías usadas se detallan a continuación:

Dart

Figura 27

Logo Dart



Lenguaje de programación desarrollado por Google. Orientado principalmente al desarrollo de aplicaciones multiplataforma, es el lenguaje usado por el framework Flutter. Permite crear aplicaciones rápidas, eficientes y fácilmente mantenibles, gracias a su fuerte tipado y sintaxis clara.

En este proyecto Dart se usará como el lenguaje de programación principal, desarrollando en él la lógica multiplataforma usando el framework Flutter. Permite implementar la UI (interfaz de Usuario, por sus siglas en inglés), gestionar el estado de la aplicación y realizar llamadas a la API para acceder a los datos del backend.

Flutter**Figura 28**

Logo Flutter



Framework de desarrollo multiplataforma creado por Google, permite el desarrollo de aplicaciones para múltiples dispositivos con una sola base de código. Usa Dart como lenguaje de programación y proporciona un gran conjunto de componentes visuales para construir interfaces modernas y responsivas.

La facilidad de este Framework es que con una sola base de código podemos trabajar tanto para móviles (iOS y Android), como para escritorio (web, Windows, etc...), lo que hace que el tiempo de desarrollo y posterior mantenimiento se vaya a reducir considerablemente, ayudando además a la coherencia visual (interfaz de usuario) entre plataformas.

Supabase**Figura 29**

Logo Supabase



Es una de las plataformas BaaS (Backend As a Service) por excelencia, funciona como base de datos, para autenticación de usuarios, almacenamiento de archivos y generación de API Rest. Basada en tecnología Open Source, como PostgreSQL.

PROMETEO

Este proyecto usará Supabase como plataforma backend cloud. ya que ofrece una solución completa donde se integran bases de datos, autenticación y almacenamiento de archivos, lo que ha permitido simplificar la arquitectura, evitando tener que desarrollar un backend completo desde 0 con otras tecnologías.

PostgreSQL

Figura 30

Logo PostgreSQL



Es un sistema de gestión de bases de datos relacionales, Open Sources, y ampliamente usado en entornos profesionales.

Es el sistema de gestión de BBDD elegido para almacenar toda la información de la aplicación, gestionada a través de Supabase, para poder tener acceso remoto y herramientas de administración adicionales. Además aporta robustez, soporta relaciones complejas y, por supuesto, es compatible con Supabase, facilitando también la integración con Backend

Git

Figura 31

Logo Git



PROMETEO

Es un sistema de control de versiones que permite gestionar los cambios de código en el proyecto a lo largo del tiempo, lo que facilita el desarrollo, colaboración y recuperación de versiones anteriores.

Se usará esta herramienta para registrar y controlar la evolución del proyecto, mediante Commits periódicos que documenten el avance y mantengan el historial de cambios.

Github

Figura 32

Logo GitHub



Plataforma online para el alojamiento de repositorios Git, para facilitar la colaboración entre desarrolladores. Ofrece además herramientas para gestionar proyectos, controlar versiones y seguir incidencias.

Se usará como repositorio remoto de almacenamiento para el código del proyecto, para poder mantener un registro del desarrollo y tener una copia segura del código.

Visual Studio Code

Figura 33

Logo Visual Studio Code



PROMETEO

Es un IDE ligero y extensible desarrollado por Microsoft. Permite trabajar con múltiples lenguajes de programación, dispone además de numerosas extensiones para facilitar el trabajo de desarrollo del software.

Será el entorno de desarrollo usado para implementar el código del proyecto, usando extensiones específicas para Flutter, Dart, Git y Supabase, mejorando así la productividad del desarrollador.

Mailtrap

Figura 34

Logo Mailtrap



Plataforma online para el testing y envío seguro de correos electrónicos en entornos de desarrollo. Permite capturar emails enviados por una aplicación sin que estos lleguen a destinatarios reales, facilitando la depuración y validación de funcionalidades relacionadas con el envío de correos. Además, ofrece herramientas para inspeccionar el contenido, cabeceras y formato de los mensajes.

Mailtrap se utilizará como entorno de pruebas para la funcionalidad de envío de correos electrónicos, en principio para confirmaciones de registro y activación de cuenta y en el futuro se testearán notificaciones al usuario. Permite verificar que los correos se generan correctamente, sin riesgo de enviar información a usuarios reales durante la fase de desarrollo, asegurando así un flujo de comunicación fiable antes de su despliegue en producción.

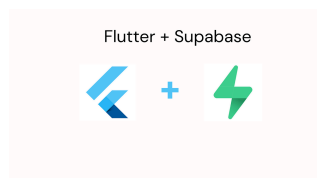
Emuladores (de dispositivos móviles).

Permiten simular el funcionamiento de diferentes dispositivos sin disponer físicamente de ellos, usados habitualmente en este tipo de proyectos.

Se usarán para probar el funcionamiento de la aplicación en los diferentes sistemas operativos y dispositivos, para poder verificar el correcto funcionamiento multiplataforma de la aplicación Flutter.

Integración Flutter + Supabase**Figura 35**

Logos Flutter y Supabase



La arquitectura Cliente-Servidor de la aplicación, se ve claramente en esta integración donde Flutter actúa como cliente y Supabase aloja el Servidor (backend), permitiendo separar la capa de interfaz de usuario de la lógica del negocio y del almacenamiento de datos, a la vez que facilita el mantenimiento y la escalabilidad.

Flutter gestiona la capa de usuario (UI y UX), y cuando éste realiza alguna acción (desde iniciar sesión, registrar vehículo hasta añadir una intervención), envía la petición al backend mediante APIs, que proporciona Supabase, además también proporciona:

- Autenticación de usuarios: Registro, inicio de sesión y control de acceso
- BBDD (PostgreSQL): Almacena toda la información de usuarios, vehículos e intervenciones.
- API REST: Automatizada, usa la estructura de de la base de datos para realizar un CRUD completo
- Almacenamiento de archivos: Para los adjuntos de las intervenciones, ya sean fotografías o documentos

PROMETEO

El flujo de funcionamiento se puede volver a ver en la Figura 1. Esta arquitectura por capas permite centralizar en la que corresponda la gestión de datos, lógica de negocio y acceso al backend y por otro lado la parte visible, de presentación y experiencia de usuario. Supabase además simplifica todo el proceso al tener servicios ya preparados, simplificando el sistema y acelerando el desarrollo del proyecto.

Metodología

Para el desarrollo del proyecto se ha adoptado una metodología incremental con enfoque ágil, basada en la división del sistema en módulos funcionales que se han ido implementando de forma progresiva. El desarrollo del proyecto tiene fases claramente definidas, que seguirán la metodología por fases:

- Análisis previo y diseño.
- Definición de requisitos.
- Diseño de diagramas.
- Desarrollo Backend.
- Desarrollo Frontend (Web y apps móviles)
- Pruebas y documentación final.

La planificación detallada y la cronografía se muestran en una figura con el diagrama de GANTT, durante la ejecución del proyecto se han producido diferentes cambios, los principales motivos de desviación de la planificación inicial han sido:

- Curva de aprendizaje de las herramientas: Supabase, Figma, etc...
- Ajustes de diseño en la base de datos: Forma de trabajo de Supabase,...
- Tiempo dedicado al desarrollo de interfaces de Figma

Diagrama de Gantt

Figura 36.1

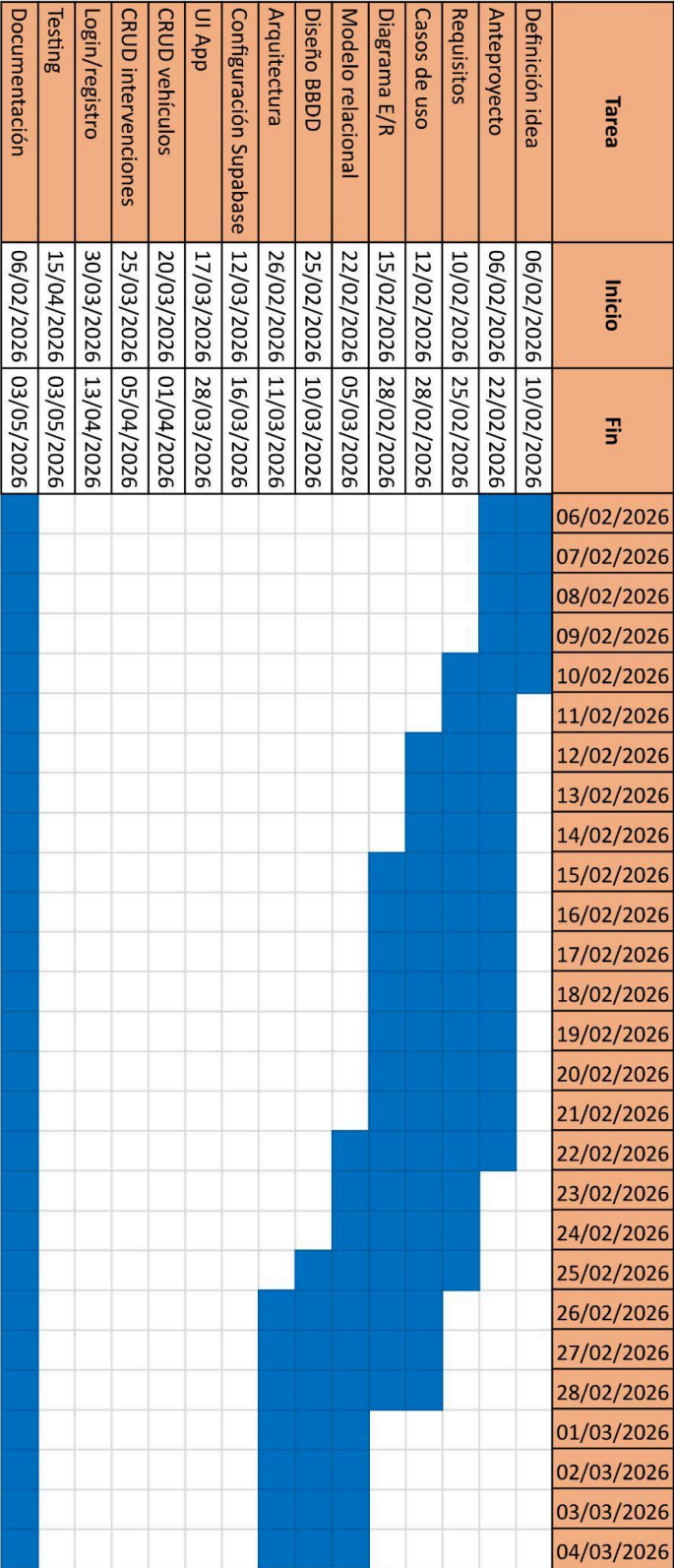


Figura 36.2

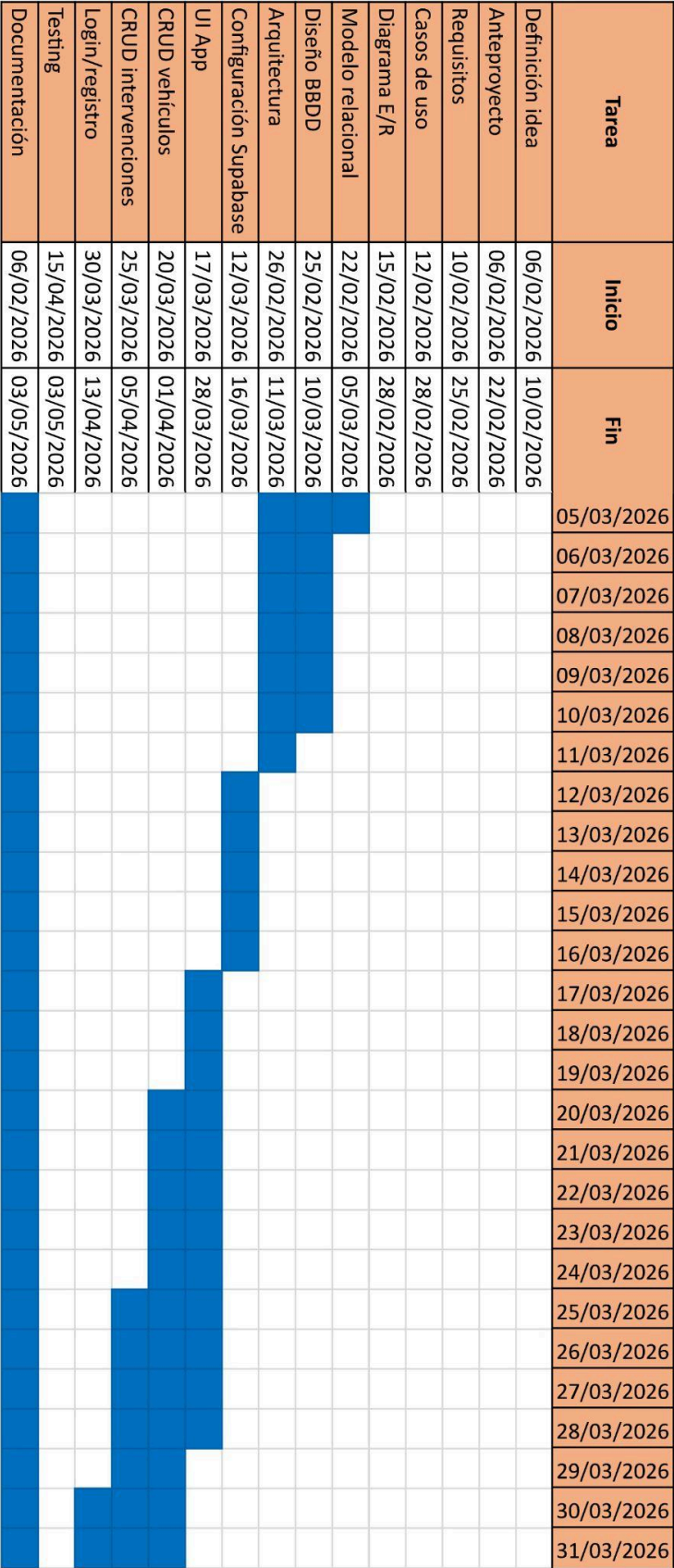


Figura 36.3

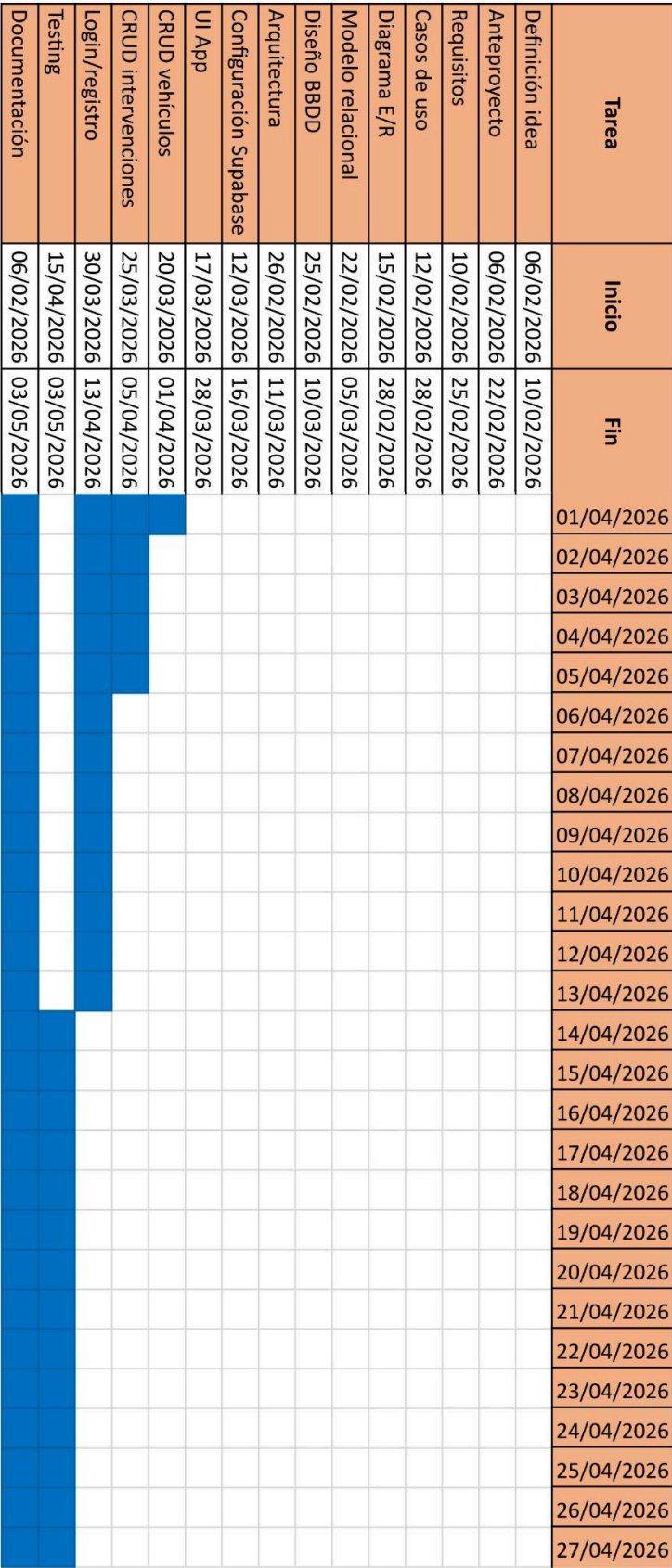


Figura 36.4

Tarea	Inicio	Fin	28/04/2026	29/04/2026	30/04/2026	01/05/2026	02/05/2026	03/05/2026
Definición idea	06/02/2026	10/02/2026						
Anteproyecto	06/02/2026	22/02/2026						
Requisitos	10/02/2026	25/02/2026						
Casos de uso	12/02/2026	28/02/2026						
Diagrama E/R	15/02/2026	28/02/2026						
Modelo relacional	22/02/2026	05/03/2026						
Diseño BBDD	25/02/2026	10/03/2026						
Arquitectura	26/02/2026	11/03/2026						
Configuración Supabase	12/03/2026	16/03/2026						
UI App	17/03/2026	28/03/2026						
CRUD vehículos	20/03/2026	01/04/2026						
CRUD intervenciones	25/03/2026	05/04/2026						
Login/registro	30/03/2026	13/04/2026						
Testing	15/04/2026	03/05/2026						
Documentación	06/02/2026	03/05/2026						

Las desviaciones mencionadas anteriormente han sido asumidas dentro del margen previsto, por lo que las fases se han completado sin afectar significativamente al resultado final, y permitiendo tener una aplicación 100% funcional.

Plan de pruebas (Testing)

El testeo de la aplicación, aunque se ha fechado para la última parte del proceso, se lleva haciendo desde que se empezó con el código, ya que se han ido realizando pruebas unitarias, para comprobar el comportamiento tanto a nivel individual (aislado) como de interrelación entre las diferentes funcionalidades. En Flutter el framework de testing viene

PROMETEO

incluido dentro del SDK por lo que se han ido ejecutando mediante el comando flutter test para verificar que todo funcione correctamente.

Una vez montada la aplicación, se han realizado tanto pruebas de caja blanca como de caja negra, las primeras enfocadas a verificar la lógica interna tanto de las funciones como de los métodos implementados en el código, las segundas evalúan el comportamiento de la aplicación desde el punto de vista del usuario. Por último también se han verificado las relaciones con servicios externos.

Pruebas Unitarias

A parte de ir probando modulo a modulo y con mensajes/salida harcodeados durante el desarrollo para ir probando modulo a modulo, este tipo de pruebas se usan en capas de utilidades, donde la funcionalidad depende únicamente de los datos introducidos sin otros efectos secundarios.

Pruebas de Caja Negra

Aquí evaluamos cómo funciona el sistema, como un todo, sin tener conocimiento de su interior, es decir, se interactúa con la aplicación como si fuera el usuario final/cliente, se introducen datos, se ejecutan acciones y se verifican que los resultados sean los esperados. Sirven principalmente para evaluar flujos de usuario, validar formularios y manejar errores.

Tabla 18

Pruebas de Caja Negra

ID	Módulo	Caso	Entrada	Esperado
CN-01	Login	Login con credenciales correctas	Email y contraseña válidos	Redirige a pantalla de inicio
CN-02	Login	Login con contraseña incorrecta	Email válido, contraseña errónea	SnackBar: error al iniciar sesión
CN-03	Login	Login con campos vacíos	Formulario vacío	No envía petición, campos sin rellenar
CN-04	Registro	Registro con datos válidos	Nombre, email y contraseñas correctas	Usuario creado, mensaje de validación de email
CN-05	Registro	Registro con contraseñas distintas	Contraseñas que no coinciden	SnackBar: las contraseñas no coinciden
CN-06	Registro	Registro con nombre con espacios	Nombre con espacios en blanco	SnackBar: el nombre no puede contener espacios
CN-07	Vehículos	Añadir vehículo con datos completos	Marca, modelo, matrícula, tipo, km, año	Vehículo creado y visible en la lista
CN-08	Vehículos	Añadir vehículo sin marca ni modelo	Formulario sin marca/modelo	SnackBar: introduce marca y modelo
CN-09	Vehículos	Editar vehículo existente	Modificar kilometraje	Datos actualizados en la ficha del vehículo
CN-10	Vehículos	Eliminar vehículo con intervenciones	Vehículo con intervenciones registradas	Vehículo e intervenciones eliminados (CASCADE)
CN-11	Vehículos	Subir imagen de vehículo	Imagen PNG < 2MB	Imagen visible en ficha y en lista

PROMETEO

CN-12	Vehículos	Subir imagen > 2MB	Imagen de gran tamaño	Imagen comprimida o mensaje de error
CN-13	Intervenciones	Registrar intervención completa	Todos los campos rellenos	Intervención visible en historial
CN-14	Intervenciones	Registrar intervención sin tipo ni lugar	Tipo y lugar vacíos	SnackBar: completa todos los campos
CN-15	Intervenciones	Filtrar por tipo Revisión	Filtro activo: Revisión	Solo se muestran intervenciones de tipo revisión
CN-16	Intervenciones	Adjuntar PDF a intervención	Archivo PDF < 2MB	Icono de clip visible en la tarjeta
CN-17	Intervenciones	Eliminar intervención	Intervención existente	Intervención eliminada del historial
CN-18	Perfil	Editar nombre de usuario	Nuevo nombre sin espacios	Nombre actualizado en modo vista
CN-19	Perfil	Cambiar contraseña con confirmación incorrecta	Contraseñas distintas	SnackBar: las contraseñas no coinciden
CN-20	Perfil	Subir foto de perfil	Imagen JPG < 2MB	Avatar actualizado en la pantalla de perfil
CN-21	Perfil	Cerrar sesión	Pulsar botón cerrar sesión y confirmar	Redirige a pantalla de login
CN-22	Conectividad	Cargar vehículos sin conexión	Servidor de base de datos inaccesible	SnackBar: sin conexión o servidor no disponible
CN-23	Conectividad	Guardar intervención con sesión expirada	Token JWT caducado	SnackBar: tu sesión ha expirado

Pruebas de Caja Blanca

Con ellas evaluamos la lógica interna del código, conociendo la implementación por lo que usamos esta lógica para diseñar las pruebas, para cubrir todos los caminos/ramas, condiciones, etc... que se ejecutan en el código. En este caso se han analizado principalmente estas funciones:

- `ErrorUtils.mensajeLegible()`: Con esta función centralizada, analizamos el mensaje del sistema con la excepción y devolvemos un texto que pueda entender el usuario.
- `FileUtils.processFile()`: Función para procesar los archivos y que decide qué hacer con ellos, comprimir, rechazar o devolver el archivo en función de extensión y tamaño.
- `filteredInterventions (getter)`: Con esta propiedades filtramos la lista de intervenciones en función de la etiqueta en base al filtro seleccionado en la UI por el usuario.
- `checkSession()`: Esta función de la `SplashScreen`, comprobamos si el usuario está ya logueado o no, para redirigirlo al home, y si no, se redirige al Login para iniciar sesión.

Tabla 19

Pruebas de Caja Blanca

ID	Función / Método	Condición probada	Resultado esperado	Verificación
CB-01	<code>_mensajeLegible()</code>	Error con "network" en mensaje	Sin conexión. Comprueba tu internet...	Detecta errores de red correctamente
CB-02	<code>_mensajeLegible()</code>	Error con "401" en mensaje	Tu sesión ha expirado...	Detecta errores de autenticación
CB-03	<code>_mensajeLegible()</code>	Error con "project is paused"	El servidor no está disponible...	Detecta Supabase pausado

CB-04	_mensajeLegible()	Error desconocido con contexto	Error al guardar el vehículo. Inténtalo...	Mensaje genérico con contexto correcto
CB-05	FileUtils.processFile()	PDF menor de 2MB	Bytes del PDF sin modificar	PDFs válidos pasan sin transformación
CB-06	FileUtils.processFile()	PDF mayor de 2MB	null	PDFs grandes son rechazados
CB-07	FileUtils.processFile()	Imagen PNG menor de 1MB	Bytes originales sin comprimir	Imágenes pequeñas no se comprimen
CB-08	FileUtils.processFile()	Imagen PNG mayor de 1MB	Bytes comprimidos al 80%	Imágenes grandes se comprimen
CB-09	filteredInterventions	Filtro "Todo" activo	Lista completa de intervenciones	Devuelve todas las intervenciones
CB-10	filteredInterventions	Filtro "Revisión" activo	Solo intervenciones con tipo "revisión"	Filtra correctamente por tipo
CB-11	checkSession()	Usuario autenticado en Supabase	Navegación a /home	Redirige al home si hay sesión activa
CB-12	checkSession()	Sin usuario autenticado	Navegación a /login	Redirige al login si no hay sesión

Pruebas de integración y sistema

Además de las pruebas descritas anteriormente se han realizado estas pruebas para comprobar que los diferentes módulos y servicios externos (Supabase Auth, Base de datos PostgreSQL y Supabase Storage) interactúan correctamente. Las pruebas han sido realizadas de forma manual, comprobando cada flujo de principio a fin:

- Registro: Rellenar Campos de registro → Validación Mail → Inicio de sesión
- Añadir imagen a vehiculo: Crear/Editar vehículo → Añadir imagen → Imagen visible en lista → Imagen visible en ficha vehículo

PROMETEO

- Editar perfil → Actualizar nombre y/o fotografía → Verificar en modo vista/Pantalla Perfil
- Eliminar vehiculo → Verificar en BDD que las intervenciones también desaparecen (ON DELETE CASCADE)
- Desconexión: Pausar proyecto en Supabase en diferentes escenarios y verificar mensajes de error legibles. Los vehiculos e intervenciones siguen siendo visibles, pero no se pueden realizar operaciones sobre ellos (edición, eliminar, añadir nuevos,...). Las páginas de perfil son visibles pero sin datos.

Presupuesto

El proyecto está estipulado en una duración de unas 50h totales de trabajo, con una estimación media de un coste de 15-20€/hora para un programador Junior, nos sale el siguiente desglose:

Tabla 20

Desglose Presupuesto estimado

Fase	Horas	Coste
Análisis	10h	150-200€
Backend	10h	150-200€
Frontend	12h	180-240€
Testing	5h	75-100€
Documentación	5h	75-100€
TOTAL	50h	750-1.000€

Trabajos Futuros

A la hora de realizar este documento se han considerado las siguientes mejoras para versiones futuras, sin estar cerrado a nuevas ideas, propias o con feedback de los usuarios:

PROMETEO

- Comprobar cumplimiento legislativo (RGPD, entre otros) antes de pasar a producción.
- Desplegar la aplicación a un entorno de producción fuera del ámbito académico para satisfacer las necesidades de los encuestados que así lo solicitaron.
- Gestión de talleres como usuarios del sistema.
- Seguimiento en tiempo real del estado de reparaciones.
- Notificaciones automáticas, avisos, recordatorios, estado reparaciones, etc....
- Firma digital avanzada.
- Integraciones con sistemas externos (fabricantes, Dirección General de Tráfico, Instituciones, Aseguradoras, etc...).

Conclusiones

Este proyecto ha permitido aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo, integrando diferentes tecnologías, métodos de trabajo, etc... en una solución final, completa y funcional.

La aplicación multiplataforma cumple con los objetivos planteados al principio del presente proyecto, ya que permite gestionar vehículos y sus intervenciones de mantenimiento. Flutter ha permitido unificar en un solo desarrollo varias plataformas (web, Android e iOS), mientras que Supabase también ha unificado la gestión de diferentes aspectos del backend (autenticación, base de datos y almacenamiento), simplificando todo el proceso de manera natural.

El proceso ha permitido valorar la importancia de todas y cada una de las fases, empezando por el análisis y diseño, con especial hincapié en la parte de definición, diagramas, estructuras y en general todo el trabajo previo a empezar a trabajar con el código en sí. Haber dado la importancia necesaria a esta fase previa ha facilitado bastante el desarrollo posterior.

PROMETEO

Algunos aspectos mejorables se han incluido en la parte de ampliaciones futuras, como la integración de talleres en el sistema, y todo lo relacionado con la gestión de documentos que estos hacen para los mantenimientos. Así como otras líneas de trabajo que se centran más en la parte de negocio que en la de programación como tal.

En conclusión, se han cumplido los objetivos planteados al principio de este proyecto, demostrando la viabilidad de la solución, la capacidad de análisis y expectativas de programación, y por tanto proporcionando una base sólida para las futuras ampliaciones mencionadas, además vista la acogida entre los 100 encuestados, también se plantea hacer despliegue en un entorno de producción real, más allá del ámbito académico.

Referencias

Arsys. (s. f.). *Todo sobre la arquitectura cliente-servidor*.

<https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>

AuTh | Supabase Docs. (2026, 3 abril). Supabase Docs.

<https://supabase.com/docs/guides/auth>

Dart documentation. (s. f.). <https://dart.dev/docs>

Documentation for Visual Studio Code. (2021, 3 noviembre).

<https://code.visualstudio.com/docs>

Figma Learn - Help Center. (s. f.). <https://help.figma.com/hc/en-us>

Flutter documentation. (s. f.). <https://docs.flutter.dev/>

Git - reference. (s. f.). <https://git-scm.com/docs>

GitHub Documentación de ayuda de .com. (s. f.). GitHub Docs.

<https://docs.github.com/es>

Gupta, L., & Gupta, L. (2025, 1 abril). *What is REST?* REST API Tutorial.

<https://restfulapi.net/>

IxDF - Interaction Design Foundation. (s. f.). *UX Design courses & Global UX Community*.

<https://ixdf.org/>

PROMETEO

Supabase Docs. (2026, 3 abril). Supabase Docs. <https://supabase.com/docs>

PostgreSQL: documentation. (s. f.). The PostgreSQL Global Development Group.

<https://www.postgresql.org/docs/>

Row Level Security | Supabase Docs. (2026, 3 abril). Supabase Docs.

<https://supabase.com/docs/guides/auth/row-level-security>

Sign in - Google Accounts. (s. f.). <https://keep.google.com/>

Software Development Life cycle. (s. f.).

https://www.tutorialspoint.com/software_engineering/software_development_life_cycle.htm

The AI workspace that works for you. | Notion. (s. f.). Notion. <https://www.notion.so/>

W3Schools.com. (s. f.). <https://www.w3schools.com/sql/>

Documentation | Mailtrap Help Center. (s. f.). Mailtrap Help Center. <https://docs.mailtrap.io/>